

3rd nov

Final year project - post quantum cryptography

Plan for initial structure

Introduction to post quantum cryptography – talk about what it is, why it is necessary and its limitations.

- quantum computers pose a significant threat to public key cryptography
- however, symmetric key cryptography only must make slight changes (increases in size etc.)

Brief discussion about harvest now, decrypt later attacks, why i find them interesting

Talk about current methods of PQC (and some of their algorithms),

- lattice based
- multivariate
- high-based
- code-based
- isogeny based
- symmetric key quantum resistance

Compare current methods of PQC

Try to create my own implementations of some PQC methods, potentially try to construct my own algorithm from one of the methods

To conclude, talk about my thoughts from research + personal attempts and conclude with opinions on how we can best prepare for a quantum safe internet.

17th nov

RSA, Diffie-Hellman, elliptical curves – vulnerable to Shor's Algorithm

Some notes on different methods of PQC:

Lattice-based cryptography:

some constructions appear to be resistant to attack by traditional and quantum computers.

Many constructions are considered secure on the assumption that certain well studied lattice problems are not efficient to solve.

18^h nov

More on lattice-based cryptography:

A little history (for background understanding):

1996, Miklos Ajtai introduced the first lattice-based construction, whose security could depend on the hardness of well-studied lattice problems.

Cynthia Dwork showed that a certain average case lattice problem, known as short integer solutions, is at least as hard to solve as a worst case lattice problem.

She then showed a cryptographic hash function whose security is equivalent to the computational hardness of SIS.

1998 Jeffrey Hoffstein, Jill Pipher and Joseph H. Silverman introduced a lattice-based public key encryption scheme (NTRU). However their scheme is not proven to be at least as hard as solving a worst case lattice problem.

2005, Oded Regev designed the first lattice-based public-key encryption scheme with proven security under worst-case hardness assumptions. (LWE)

A lot of follow up work on Regev's security proof, improving efficiency of the scheme

Work has been devoted to constructing additional cryptographic primitives based on LWE.

2009, Craig Gentry introduced the first fully homomorphic encryption (allows computations on data without first having to decrypt the data) scheme that was based on a lattice problem.

Mathematical foundations:

Upon initially looking at the explanation of a lattice, I felt confused as a lattice is a new concept to me, so I have decided to delve more into the actual mathematical side of a lattice, and then swing it round into understanding how it can be applied to cryptography.

Source, Tel Aviv uni lecture on Lattices given by Oded Regev:

Defintion of a lattice:

Given n linearly independent vectors $b_1, b_2, \dots, b_n$ belonging to $\mathbb{R}^m$, the lattice generated by them is defined as:

$$\mathcal{L}(b_1, b_2, \dots, b_n) = \left\{ \sum x_i b_i \mid x_i \in \mathbb{Z} \right\}.$$

We refer to $b_1, b_2, \dots, b_n$ as the basis of the lattice. Equivalently, if we define B as the $m \times n$ matrix whose columns are $b_1, b_2, \dots, b_n$ then the lattice generated by B is

$$\mathcal{L}(B) = \mathcal{L}(b_1, b_2, \dots, b_n) = \{ Bx \mid x \in \mathbb{Z}^n \}.$$

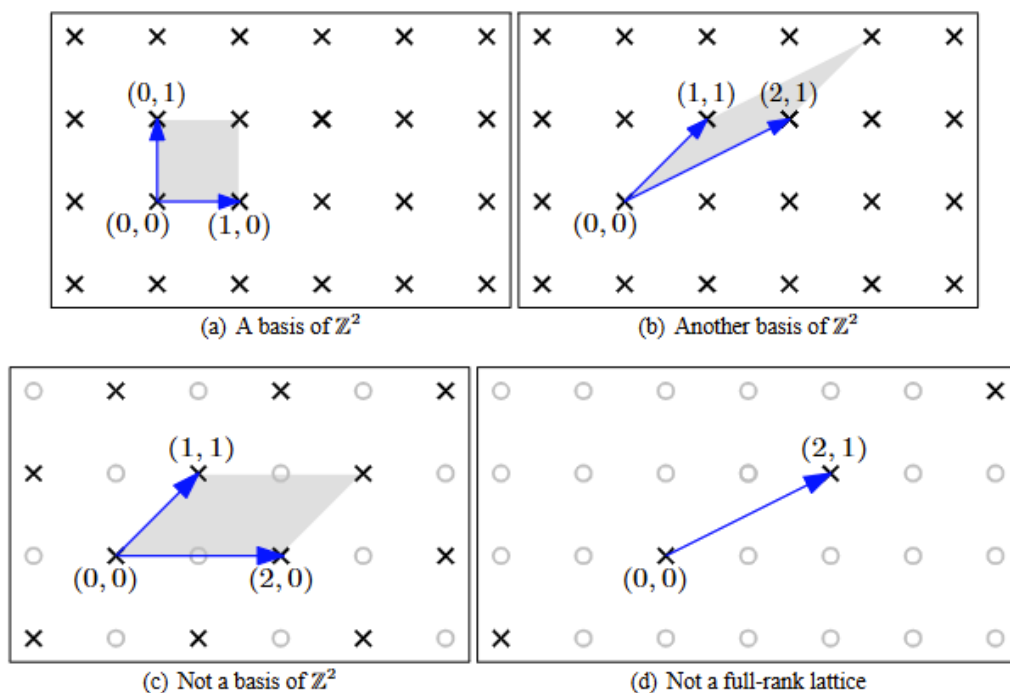


Figure 2: Some lattice bases

I now feel as though my understanding of a lattice is concrete, however i will refer to the source if I have remaining, un-answered questions.

(back to wiki)

curtiucally, the basis for a lattice is not unique – and from initial intuition, I believe that this is where its cryptographic value comes from.

The most important lattice-based computational problem is the shortest vector problem.

2nd december

My intuition, of how lattice-based cryptography works that we encrypt our data with the basis of the lattice as the “key”, to a “ciphertext” which is our lattice. It is now hard to find which basis was used to encrypt the data, and hence it is difficult to find the “plaintext” from the “ciphertext”

From a discussion with my academic tutor, we concluded that the “quantum safe” elements of lattice-based cryptography lie in the fact that each lattice has an infinite number of bases. What makes a quantum computer dangerous for encryption, is its parallel capabilities. However, this also relies on the solution being of a known finite number of possibilities. (exact reason for this I feel right now I will struggle to put this into words). Since our lattice has infinite basis, our ciphertext will hence have infinite keys, meaning that it will be incredibly complicated and potentially impossible to solve. However, on the other hand if it is possible to isolate a certain section of our lattice, we will end up with finite bases, and hence lattice cryptography will not be secure.

Furthermore, I believe that lattice-based cryptography will have potential for implementation. I will now do further research into quantum computing, the fundamentals of quantum cryptography, and some current lattice-based algorithms to support or redirect my intuition.

Pre-amble

Quantum Computers.

Quantum Computers can operate on an enormous number of possibilities simultaneously, though still subject to strict computational constraints. (Non-deterministic)

It is widely believed that Quantum computers could perform some calculations exponentially faster than any classical computer. Such as widely used public-key cryptographic schemes as well as help physicists with physical simulations. (the latter will not be discussed in this project)

The basic unit of information in a quantum computer is known as a qubit, it serves the same function as a “bit” in a classical computer.

However, unlike a classic bit which can only exist in 2 states, a qubit can exist in a linear combination of 2 states, known as a quantum superposition.

Quantum superposition is a fundamental principle of quantum mechanics, that states that any linear combination of solutions to the Schrodinger equation are also solutions

to the Schrodinger equation. The state of a system is given by a linear combination of all the eigenfunctions of the Schrodinger equation governing that system.

The state of a qubit is most generally a superposition of the two basis states $\text{ket}(0)$ and $\text{ket}(1)$.

$$|\Psi\rangle = c_0|0\rangle + c_1|1\rangle,$$

Where the left-hand side is the quantum state of the qubit and $\text{ket}(0)$ and $\text{ket}(1)$ denote solutions to the schrodinger equation, weighted by the two probability amplitudes c_0 and c_1 , which are both complex numbers.

Quantum Cryptography (this is something i may research more later, as i think that discussing / implementing a handshaking algorithm would be advantageous)

Quantum key distribution -

Post-quantum cryptography

Post-quantum cryptography is the development of cryptographic algorithms that are resistant to a cryptanalytic attack by a quantum computer.

A cryptanalytic attack involves analysing security systems, to understand the hidden aspects of the system to breach the system, and gain access to encrypted messages with having access to the encryption key.

Most widely used public key algorithms, rely on the 3-following mathematical problems: integer factorisation problem, the discrete logarithm problem and the elliptical curve problem.

However, on a sufficiently powerful quantum computer with Shor's algorithm, these problems can be easily solved.

Shor's algorithm is a quantum algorithm for finding the prime factors of an integer.

It was developed in 1994 by peter Shor and remains one of the few quantum algorithms with compelling applications and strong evidence of super polynomial speedup compared to the best-known classic algorithms.

<https://academic.oup.com/book/41807/chapter-abstract/354545827?redirectedFrom=fulltext&login=true>

However, beating classic computers will require millions of qubits due to the overhead caused by quantum error correction. (Quantum error correction, comprises of a set of techniques used in quantum memory and quantum computing to protect quantum information from errors arising from decoherence and other sources of quantum noise.)

Shor proposed multiple similar algorithms, for solving the factoring problem, the discrete logarithm problem and the period-finding problem.

On a quantum computer, to factor an integer, Shor's algorithm runs in polynomial time. (meaning the time taken is polynomial in $\log N$??)

Shor's algorithm is asymptotically faster than the most scalable classic algorithm, the general number field sieve, which works in sub-exponential time.

Shor's algorithm, given a quantum computer with enough qubits and without succumbing to quantum noise, would be able to break the following cryptographic schemes:

RSA, Diffie-Hellman key exchange and elliptical-curve Diffie Hellman.

19th Jan

Qubit is a wave with two modes, which when measured collapses to a particle. Before measurement, we have coefficients which determine the probability of each measurement.

What is the mode of a wave?

I think i understand how it works logically, we have the superposition of the qubit, which has probability amplitudes, which are coefficients to each state, and when squared determine the probability of the likelihood of measurement of a state. Which gives us 2^n information of a classical bit. Until measurement, at which case its state is determined, we "lose" the superposition. I'm still uncertain on the actual quantum mechanics, and I don't think it would be necessary for the project, but I'm still curious on how it would work.

Quantum computers are faster for certain calculations where you can make use of the fact that you have all the quantum superpositions available at the same time, to do some kind of computational parallelism.

A computer where the number of computations required to arrive at the result is exponentially smaller than a classic computer. - not universally the case, only in certain calculations or algorithms. (shors algorithm for example)

Brief delve into different post-quantum crypto methods (exisiting algorithms and implementations)

Lattice based:
Cystals kyber – there exists implementations of this algorithm (standardised by NIST).
Key encapsulation algorithm. Already used in google, apple, cloudflare, linux projects.

Multi-variate:
Rainbow signature scheme – most attempts of multivariate have failed, however rainbow could provide a basis for a quantum secure digital signature.

Hash based:
XMSS – mention in RFC 8391 (stateful hash-based signature scheme)

Code based:
McEliece public key encryption scheme – suggested by the post-quantum cryptography group as a candidate for long term protection against quantum era attacks. It has withstood scrutiny for 40+ years.

Isogeny-based cryptography:
diffie-hellman-like key exchange can serve as simple quantum-resistant replacement for current in-use diffie-hellman key-exchange methods.

SIDH/SIKE was spectacularly broken in 2022 – seems like an interesting point of research if i have more time.

Symmetric key quantum resistance

With sufficiently large key sizes, the symmetric key crypto systems, like AES and SNOW 3G are already resistant to attack by quantum computer.

Since it currently has widespread deployment, some researchers recommend expanded use of kerberos-like symmetric key management as an efficient way to get post-quantum cryptography today.

Key exchanged/ KEM – securely agree on a shared secret key over an insecure network, KEM is a method of key exchange

Signature scheme – prove authenticity, integrity or origin of data

A cryptographic scheme is a combination of multiple primitives working together to provide security goals

KEM – primitive

Signature scheme – primitive

Key exchanged – security goal

2nd feb

Crystals kyber – learning with errors problem.

LWE – take a linear equation involving s and add a tiny amount of random noise – now becomes strikingly difficult to solve.

<https://pq-crystals.org/kyber/>

Ring polynomial -

Let R be a ring.

The **polynomial ring over R in one indeterminate x** , denoted $R[x]$, is the set

$$R[x] = \left\{ \sum_{i=0}^n a_i x^i \mid n \in \mathbb{N}, a_i \in R \right\}$$

with operations defined by

$$\sum_{i=0}^n a_i x^i + \sum_{i=0}^m b_i x^i = \sum_{i=0}^{\max(n,m)} (a_i + b_i) x^i$$

$$\left(\sum_{i=0}^n a_i x^i \right) \left(\sum_{j=0}^m b_j x^j \right) = \sum_{k=0}^{n+m} \left(\sum_{i+j=k} a_i b_j \right) x^k$$

A **vector space** is a set whose elements can be added and scaled by elements of a **field**.

A **ring** allows addition and multiplication of its own elements.

A **field** is a ring where every nonzero element has a multiplicative inverse.

Two implementations

Ref – in C, much slower but designed around readability, portability and correctness. Verifies algorithms behaviour but is slower, can be used on any processor. Used for testing and validation.

Avx2, much faster, however can only work on processors that support avx2. Used for deployment on modern servers and desktops. It uses vector instructions to process multiple data points simultaneously, offering significant speed improvements in lattice-based polynomial arithmetic (NTT), compression, and packing. Uses c but also avx2 intrinsics or assembly for optimisation of vectorised operations

Please note that the reference implementation in `ref/` is not optimized for any platform, and, since it prioritises clean code, is significantly slower than a trivially optimized but still platform-independent implementation. Hence benchmarking the reference code does not provide particularly meaningful results.

<https://github.com/pq-crystals/kyber/tree/main>

I have managed to get the working implementation of kyber-crystals working

Goals for implementation:

Lattice-based encryption algorithm

Realistically, “proving security” is too complicated for the project, and sort of extends the scope. (make some attempt of a loose “proof”), with use of base case

To design, and then implement a key-exchange (like kyber)

Share a key, between 2 clients, encrypt, decrypt, output this is in a “relatively” clean format.

Initially I’m going to write it in python, for comfort, and then switch to C/C++ for a speed up (if time allows).

Notes on Kyber - “base case” implementation.

Won first post-quantum cryptography standard competition held by NIST.

Asymmetric cryptosystem.

Key-encapsulation method. (IND-CCA2) attacker.

LWE-lattice problem

There exists a complementary signature scheme to kyber, dilithium, could lend a hand to potential implementation of a signature scheme to create a full cryptography scheme. (if time allows)

<https://cryptopedia.dev/posts/kyber/>

chrome-

extension://oemmndcbldboiebfnladdacbfdmadadm/https://eprint.iacr.org/2022/472.pdf

chrome-

extension://oemmndcbldboiebfnladdacbfdmadadm/https://cims.nyu.edu/~regev/papers/lwesurvey.pdf